23MID0233

GILLEDI SANTHOSH

ADVANCE PYTHON PROGRAMMING LAB ASSIGNMENT

LOAN APPROVAL SYSTEM

In [116…
```
import pandas as pd
from sklearn.metrics import accuracy_score
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, f1_score
from sklearn.model_selection import train_test_split
from sklearn.metrics import ConfusionMatrixDisplay
```

In [96]:
```
df=pd.read_csv("loan_datset.csv")
print(df.head())
```

```
   Customer_ID  Gender Married  Dependents     Education Self_Employed  \
0          569  Female      No           0      Graduate            No
1           15    Male     Yes           2      Graduate            No
2           95    Male      No           0  Not Graduate            No
3          134    Male     Yes           0      Graduate           Yes
4          556    Male     Yes           1      Graduate            No

   Applicant_Income  Coapplicant_Income  Loan_Amount  Loan_Amount_Term  \
0              2378                 0.0            9               360
1              1299              1086.0           17               120
2              3620                 0.0           25               120
3              3459                 0.0           25               120
4              5468              1032.0           26               360

   Credit_History Property_Area Loan_Status
0               1         Urban           N
1               1         Urban           Y
2               1     Semiurban           Y
3               1     Semiurban           Y
4               1     Semiurban           Y
```

In [97]:
```
print(df.columns)
```

```
Index(['Customer_ID', 'Gender', 'Married', 'Dependents', 'Education',
       'Self_Employed', 'Applicant_Income', 'Coapplicant_Income',
       'Loan_Amount', 'Loan_Amount_Term', 'Credit_History', 'Property_Are
a',
       'Loan_Status'],
      dtype='str')
```

In [98]:
```
print(df.shape)
```

```
(614, 13)
```

In [99]:
```
print(df.isnull().sum())
```

```
Customer_ID           0
Gender                0
Married               0
Dependents            0
Education             0
Self_Employed         0
Applicant_Income      0
Coapplicant_Income    0
Loan_Amount           0
Loan_Amount_Term      0
Credit_History        0
Property_Area         0
Loan_Status           0
dtype: int64
```

In [100…
```
# Remove duplicates
df = df.drop_duplicates()
```

In [101…
```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 614 entries, 0 to 613
Data columns (total 13 columns):
 #   Column              Non-Null Count  Dtype  
---  ------              --------------  -----  
 0   Customer_ID         614 non-null    int64  
 1   Gender              614 non-null    str    
 2   Married             614 non-null    str    
 3   Dependents          614 non-null    int64  
 4   Education           614 non-null    str    
 5   Self_Employed       614 non-null    str    
 6   Applicant_Income    614 non-null    int64  
 7   Coapplicant_Income  614 non-null    float64
 8   Loan_Amount         614 non-null    int64  
 9   Loan_Amount_Term    614 non-null    int64  
 10  Credit_History      614 non-null    int64  
 11  Property_Area       614 non-null    str    
 12  Loan_Status         614 non-null    str    
dtypes: float64(1), int64(6), str(6)
memory usage: 77.8 KB
```

In [102…
```
df.describe()
```

Out[102…

	Customer_ID	Dependents	Applicant_Income	Coapplicant_Income	Loan_Amoun
count	614.000000	614.000000	614.000000	614.000000	614.00000
mean	307.500000	0.856678	5403.459283	1621.245798	142.02280
std	177.390811	1.216651	6109.041673	2926.248369	87.08308
min	1.000000	0.000000	150.000000	0.000000	9.00000
25%	154.250000	0.000000	2877.500000	0.000000	98.00000
50%	307.500000	0.000000	3812.500000	1188.500000	125.00000
75%	460.750000	2.000000	5795.000000	2297.250000	164.75000
max	614.000000	4.000000	81000.000000	41667.000000	700.00000

In [103…
```
df.describe(include='object')
```

```
/tmp/ipykernel_23102/2262070296.py:1: Pandas4Warning: For backward compati
bility, 'str' dtypes are included by select_dtypes when 'object' dtype is
specified. This behavior is deprecated and will be removed in a future ver
sion. Explicitly pass 'str' to `include` to select them, or to `exclude` t
o remove them and silence this warning.
See https://pandas.pydata.org/docs/user_guide/migration-3-strings.html#str
ing-migration-select-dtypes for details on how to write code that works wi
th pandas 2 and 3.
  df.describe(include='object')
```

Out[103…

	Gender	Married	Education	Self_Employed	Property_Area	Loan_Status
count	614	614	614	614	614	614
unique	2	2	2	2	3	2
top	Male	Yes	Graduate	No	Semiurban	Y
freq	499	399	480	523	233	422

In [104…
```
df.dtypes
```

Out[104…
```
Customer_ID            int64
Gender                   str
Married                  str
Dependents             int64
Education                str
Self_Employed            str
Applicant_Income       int64
Coapplicant_Income   float64
Loan_Amount            int64
Loan_Amount_Term       int64
Credit_History         int64
Property_Area            str
Loan_Status              str
dtype: object
```

In [105…
```
(df.isnull().mean() * 100).round(2)
```

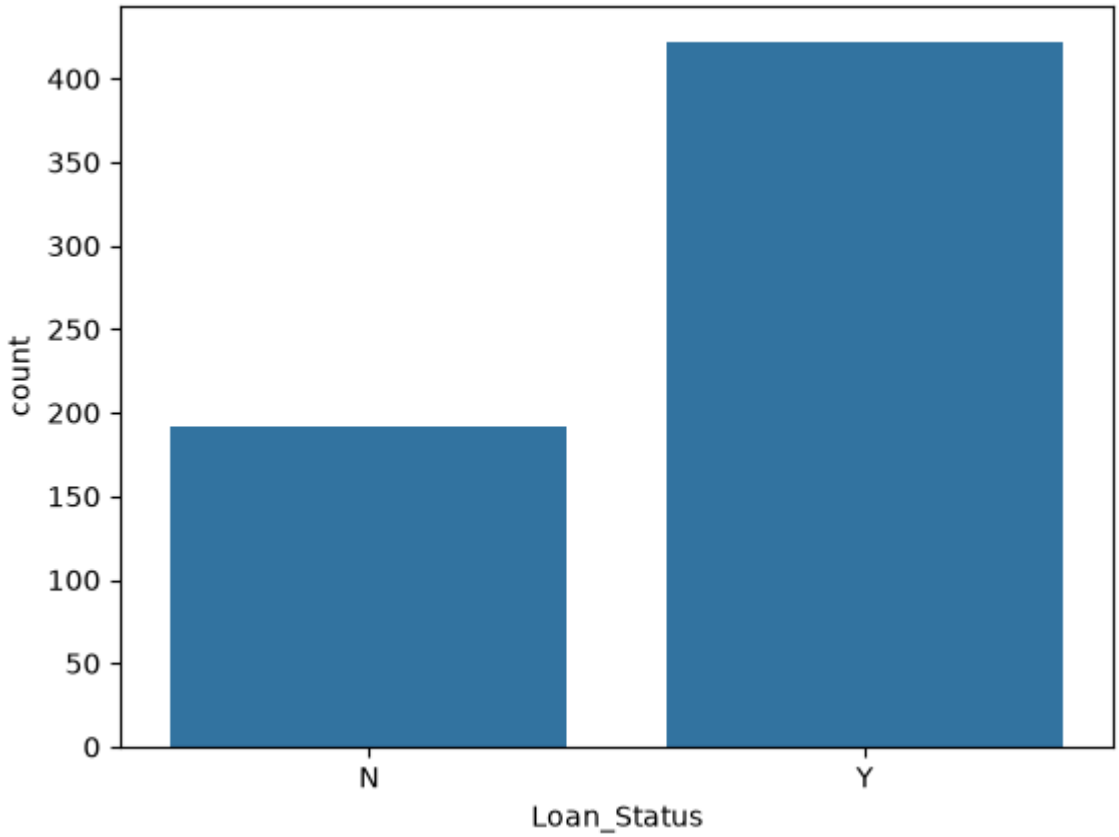
Out[105…
```
Customer_ID          0.0
Gender               0.0
Married              0.0
Dependents           0.0
Education            0.0
Self_Employed        0.0
Applicant_Income     0.0
Coapplicant_Income   0.0
Loan_Amount          0.0
Loan_Amount_Term     0.0
Credit_History       0.0
Property_Area        0.0
Loan_Status          0.0
dtype: float64
```

In [106…
```
df.duplicated().sum()
```

Out[106…
```
np.int64(0)
```

In [107…
```
import matplotlib.pyplot as plt
import seaborn as sns
print(df['Loan_Status'].value_counts())
print(df['Loan_Status'].value_counts(normalize=True))
sns.countplot(x='Loan_Status', data=df)
plt.show()
```

```
Loan_Status
Y    422
N    192
Name: count, dtype: int64
Loan_Status
Y    0.687296
N    0.312704
Name: proportion, dtype: float64
```



In [ ]:

In [108…
```
train_ids = df['Applicant_Income']
train = df.drop('Applicant_Income', axis=1)

# Categorical: mode
for col in ['Gender', 'Married', 'Dependents', 'Self_Employed', 'Credit_H
    train[col] = df[col].fillna(train[col].mode()[0])

# Numerical: median
train['LoanAmount'] = df['LoanAmount'].fillna(train['LoanAmount'].median(
train['Loan_Amount_Term'] = df['Loan_Amount_Term'].fillna(train['Loan_Amo

print(df.isnull().sum())
```

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
File ~/.local/lib/python3.13/site-packages/pandas/core/indexes/base.py:3641, in Index.get_loc(self, key)
   3640 try:
-> 3641     return self._engine.get_loc(casted_key)
   3642 except KeyError as err:

File pandas/_libs/index.pyx:168, in pandas._libs.index.IndexEngine.get_loc()

File pandas/_libs/index.pyx:197, in pandas._libs.index.IndexEngine.get_loc()

File pandas/_libs/hashtable_class_helper.pxi:7668, in pandas._libs.hashtable.PyObjectHashTable.get_item()

File pandas/_libs/hashtable_class_helper.pxi:7676, in pandas._libs.hashtable.PyObjectHashTable.get_item()

KeyError: 'LoanAmount'

The above exception was the direct cause of the following exception:

KeyError                                  Traceback (most recent call last)
Cell In[108], line 9
      6     train[col] = df[col].fillna(train[col].mode()[0])
      8 # Numerical: median
----> 9 train['LoanAmount'] = df['LoanAmount'].fillna(train['LoanAmount'].median())
     10 train['Loan_Amount_Term'] = df['Loan_Amount_Term'].fillna(train['Loan_Amount_Term'].mode()[0])
     12 print(df.isnull().sum())

File ~/.local/lib/python3.13/site-packages/pandas/core/frame.py:4378, in DataFrame.__getitem__(self, key)
   4376 if self.columns.nlevels > 1:
   4377     return self._getitem_multilevel(key)
-> 4378 indexer = self.columns.get_loc(key)
   4379 if is_integer(indexer):
   4380     indexer = [indexer]

File ~/.local/lib/python3.13/site-packages/pandas/core/indexes/base.py:3648, in Index.get_loc(self, key)
   3643     if isinstance(casted_key, slice) or (
   3644         isinstance(casted_key, abc.Iterable)
   3645         and any(isinstance(x, slice) for x in casted_key)
   3646     ):
   3647         raise InvalidIndexError(key) from err
-> 3648     raise KeyError(key) from err
   3649 except TypeError:
   3650     # If we have a listlike key, _check_indexing_error will raise
   3651     #  InvalidIndexError. Otherwise we fall through and re-raise
   3652     #  the TypeError.
   3653     self._check_indexing_error(key)

KeyError: 'LoanAmount'
```

In [109…
```
X_train, X_test, y_train, y_test = train_test_split(
 X,
 y,
 test_size=0.2,
 random_state = 42
)
print(f"Training set size: {len(X_train)} samples")
print(f"Testing set size: {len(X_test)} samples")
```

```
Training set size: 491 samples
Testing set size: 123 samples
```

In [110…
```
df['Dependents'] = df['Dependents'].replace('3+', 3).astype(int)

df['Gender'] = df['Gender'].map({'Male': 1, 'Female': 0})
df['Married'] = df['Married'].map({'Yes': 1, 'No': 0})
df['Education'] = df['Education'].map({'Graduate': 1, 'Not Graduate': 0})
df['Self_Employed'] = df['Self_Employed'].map({'Yes': 1, 'No': 0})
df['Loan_Status'] = df['Loan_Status'].map({'Y': 1, 'N': 0})


df.head()
```

Out[110…

	Customer_ID	Gender	Married	Dependents	Education	Self_Employed	Applicant_
0	569	0	0	0	1	0	
1	15	1	1	2	1	0	
2	95	1	0	0	0	0	
3	134	1	1	0	1	1	
4	556	1	1	1	1	0	

In [111…
```
train_ids = df['Loan_ID']
train = train.drop('Loan_ID', axis=1)

# Categorical: mode
for col in ['Gender', 'Married', 'Dependents', 'Self_Employed', 'Credit_H
    train[col] = df[col].fillna(train[col].mode()[0])

# Numerical: median
train['LoanAmount'] = df['LoanAmount'].fillna(train['LoanAmount'].median(
train['Loan_Amount_Term'] = df['Loan_Amount_Term'].fillna(train['Loan_Amo

print(df.isnull().sum())
```

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
File ~/.local/lib/python3.13/site-packages/pandas/core/indexes/base.py:3641, in Index.get_loc(self, key)
   3640 try:
-> 3641     return self._engine.get_loc(casted_key)
   3642 except KeyError as err:

File pandas/_libs/index.pyx:168, in pandas._libs.index.IndexEngine.get_loc()

File pandas/_libs/index.pyx:197, in pandas._libs.index.IndexEngine.get_loc()

File pandas/_libs/hashtable_class_helper.pxi:7668, in pandas._libs.hashtable.PyObjectHashTable.get_item()

File pandas/_libs/hashtable_class_helper.pxi:7676, in pandas._libs.hashtable.PyObjectHashTable.get_item()

KeyError: 'Loan_ID'

The above exception was the direct cause of the following exception:

KeyError                                  Traceback (most recent call last)
Cell In[111], line 1
----> 1 train_ids = df['Loan_ID']
      2 train = train.drop('Loan_ID', axis=1)
      4 # Categorical: mode

File ~/.local/lib/python3.13/site-packages/pandas/core/frame.py:4378, in DataFrame.__getitem__(self, key)
   4376 if self.columns.nlevels > 1:
   4377     return self._getitem_multilevel(key)
-> 4378 indexer = self.columns.get_loc(key)
   4379 if is_integer(indexer):
   4380     indexer = [indexer]

File ~/.local/lib/python3.13/site-packages/pandas/core/indexes/base.py:3648, in Index.get_loc(self, key)
   3643     if isinstance(casted_key, slice) or (
   3644         isinstance(casted_key, abc.Iterable)
   3645         and any(isinstance(x, slice) for x in casted_key)
   3646     ):
   3647         raise InvalidIndexError(key) from err
-> 3648     raise KeyError(key) from err
   3649 except TypeError:
   3650     # If we have a listlike key, _check_indexing_error will raise
   3651     #  InvalidIndexError. Otherwise we fall through and re-raise
   3652     #  the TypeError.
   3653     self._check_indexing_error(key)

KeyError: 'Loan_ID'
```

In [118…

```
from sklearn.tree import DecisionTreeClassifier
model = DecisionTreeClassifier(
 criterion='entropy',
```

```
 max_depth=4,
 random_state=42
)
print("Decision Tree model trained successfully!")
```

```
Decision Tree model trained successfully!
```

In [119…

```
model.fit(X_train, y_train)
y_pred_dt = model_dt.predict(X_test)
accuracy_dt = accuracy_score(y_test, y_pred_dt)
f1_dt = f1_score(y_test, y_pred_dt, average='macro')
print(f"\nDecision Tree Accuracy: {accuracy_dt*100:.2f}%")
print("Decision Tree F1 Score:", f1_dt)
cm_dt = confusion_matrix(y_test, y_pred_dt)
print("Decision Tree Confusion Matrix:")
print(cm_dt)
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
/tmp/ipykernel_23102/3142672705.py in ?()
----> 1 model.fit(X_train, y_train)
      2 y_pred_dt = model_dt.predict(X_test)
      3 accuracy_dt = accuracy_score(y_test, y_pred_dt)
      4 f1_dt = f1_score(y_test, y_pred_dt, average='macro')

~/.local/lib/python3.13/site-packages/sklearn/base.py in ?(estimator, *args, **kwargs)
   1399                     )
   1400                 ),
   1401                 callback_management_context(estimator),
   1402             ):
-> 1403                 return fit_method(estimator, *args, **kwargs)

~/.local/lib/python3.13/site-packages/sklearn/tree/_classes.py in ?(self, X, y, sample_weight, check_input)
   1014         self : DecisionTreeClassifier
   1015             Fitted estimator.
   1016         """
   1017
-> 1018         super()._fit(
   1019             X,
   1020             y,
   1021             sample_weight=sample_weight,

~/.local/lib/python3.13/site-packages/sklearn/tree/_classes.py in ?(self, X, y, sample_weight, check_input, missing_values_in_feature_mask)
    243             check_X_params = dict(
    244                 dtype=np.float32, accept_sparse="csc", ensure_all_finite=False
    245             )
    246             check_y_params = dict(ensure_2d=False, dtype=None)
--> 247             X, y = validate_data(
    248                 self, X, y, validate_separately=(check_X_params, check_y_params)
    249             )
    250

~/.local/lib/python3.13/site-packages/sklearn/utils/validation.py in ?(_estimator, X, y, reset, validate_separately, skip_check_array, **check_params)
   3046             # :(
   3047             check_X_params, check_y_params = validate_separately
   3048             if "estimator" not in check_X_params:
   3049                 check_X_params = {**default_check_params, **check_X_params}
-> 3050             X = check_array(X, input_name="X", **check_X_params)
   3051             if "estimator" not in check_y_params:
   3052                 check_y_params = {**default_check_params, **check_y_params}
   3053             y = check_array(y, input_name="y", **check_y_params)

~/.local/lib/python3.13/site-packages/sklearn/utils/validation.py in ?(array, accept_sparse, accept_large_sparse, dtype, order, copy, force_writeable, ensure_all_finite, ensure_non_negative, ensure_2d, allow_nd, ensure_min_samples, ensure_min_features, estimator, input_name)
```

```
   1032                         )
   1033                     array = xp.astype(array, dtype, copy=False)
   1034                 else:
   1035                     array = _asarray_with_order(array, order=orde
r, dtype=dtype, xp=xp)
-> 1036             except ComplexWarning as complex_warning:
   1037                 raise ValueError(
   1038                     "Complex data not supported\n{}\n".format(arra
y)
   1039                 ) from complex_warning

~/.local/lib/python3.13/site-packages/sklearn/utils/_array_api.py in ?(arr
ay, dtype, order, copy, xp, device)
    972         # Use NumPy API to support order
    973         if copy is True:
    974             array = numpy.array(array, order=order, dtype=dtype)
    975         else:
--> 976             array = numpy.asarray(array, order=order, dtype=dtype)
    977
    978         # At this point array is a NumPy ndarray. We convert it to
an array
    979         # container that is consistent with the input's namespace.

~/.local/lib/python3.13/site-packages/pandas/core/generic.py in ?(self, dt
ype, copy)
   2023             )
   2024         values = self._values
   2025         if copy is None:
   2026             # Note: branch avoids `copy=None` for NumPy 1.x suppor
t
-> 2027             arr = np.asarray(values, dtype=dtype)
   2028         else:
   2029             arr = np.array(values, dtype=dtype, copy=copy)
   2030

ValueError: could not convert string to float: 'Female'
```

In [120…

```
# 1. Train your model and generate predictions
# (Replace 'model' with the actual name of your trained Decision Tree var
y_pred_dt = model.predict(x_test)

# 2. Display the confusion matrix
ConfusionMatrixDisplay.from_predictions(
    y_test,
    y_pred_dt,
    display_labels=iris.target_names,
    cmap='Greens'
)
plt.title("Decision Tree Confusion Matrix")
plt.show()
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[120], line 3
      1 # 1. Train your model and generate predictions
      2 # (Replace 'model' with the actual name of your trained Decision Tree variable)
----> 3 y_pred_dt = model.predict(x_test)
      5 # 2. Display the confusion matrix
      6 ConfusionMatrixDisplay.from_predictions(
      7     y_test,
      8     y_pred_dt,
      9     display_labels=iris.target_names,
     10     cmap='Greens'
     11 )

NameError: name 'x_test' is not defined
```

In [ ]:

In [115…

```
results = []
for name, model in models.item():
    Xtr, Xv = (X_train_sc, X_val_sc) if name in scaled_models else (X_tra
    model.fit(Xtr, y_train)
    pred = model.predict(Xv)
    results.append({
        'Model': name,
        'Accuracy': accuracy_score(y_val, pred),
        'Precision': precision_score(y_val, pred),
        'Recall': recall_score(y_val, pred),
        'F1': f1_score(y_val, pred),
    })

results_df = pd.DataFrame(results).sort_values('Accuracy', ascending=Fals
print(results_df)
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
Cell In[115], line 2
      1 results = []
----> 2 for name, model in models.item():
      3     Xtr, Xv = (X_train_sc, X_val_sc) if name in scaled_models else (X_train, X_val)
      4     model.fit(Xtr, y_train)

AttributeError: 'DecisionTreeClassifier' object has no attribute 'item'
```

In [ ]:

In [ ]: